

Object Oriented Programming

Java



Inheritance

- Inheritance allows the creation of a subclass from an existing superclass.
- When it is necessary to derive **many children** from a parent class, a number of subclasses can be inherited in java.
- But sometimes it is necessary to derive a subclass from **several classes**, thus inheriting their data and methods. *In Java only one parent class is allowed.*
- **With interfaces**, you can obtain effect of **multiple inheritance**.



What is an Interface?

- An interface is a class that contains only **constants** and **abstract methods**.
- Cannot be instantiated. Only classes that implements interfaces can be instantiated.
- *Why not just use abstract classes?*

Java does not permit multiple inheritance from classes, but permits **implementation of multiple interfaces**.



What is an Interface?

- One class can implement many interfaces.
- One interface can be implemented by many classes.
- By providing the **interface** keyword, Java allows you to fully utilize the "**one interface, multiple methods**" aspect of polymorphism.



Creating an Interface

```
public interface InterfaceClass {  
    public void method1();  
    public void method2();  
    public void method3();  
}
```

```
public class ExampleClass implements InterfaceClass {  
    public void method1(){...}  
    public void method2(){...}  
    public void method3(){...}  
}
```



Interface-Rules

- To implement an interface class '**implements**' keyword has to be used.
- An implemented class has to **override all the methods** of the interface class.
- If a class implements multiple interfaces, it should override all the abstract methods declared in all the interfaces.

```
public class ClassName implements InterfaceName1, InterfaceName2, ..., InterfaceNameN
{
    methods1 implementation;
    methods2 implementation;
    ...
    methodsN implementation;
}
```



Polymorphism

- In Java, we can create a class and declare variables, methods which are bound together. This is known as **data binding**.
- Two ways of binding in java:
 - **Static or early binding**
 - Occurs at **compile-time**. Method with the keyword static, final, or private in front of its name, binding is going to be static.
 - **Dynamic or late binding**
 - Occurs at **run-time** because Java won't necessarily know which method/field to reference at compile time.



Polymorphism

- In Java, polymorphism refers to the **dynamic binding** mechanism that determines which method definition will be used when a method name has been overridden.
- Thus, polymorphism refers to dynamic binding.
- Can treat an object of a subclass as an object of its superclass.
 - *A reference variable of a superclass type can point to an object of its subclass.*



Polymorphism

```
public class Person{
    public String type = "Person";
    public void getType(){
        System.out.println("This is a "+type);
    }
}

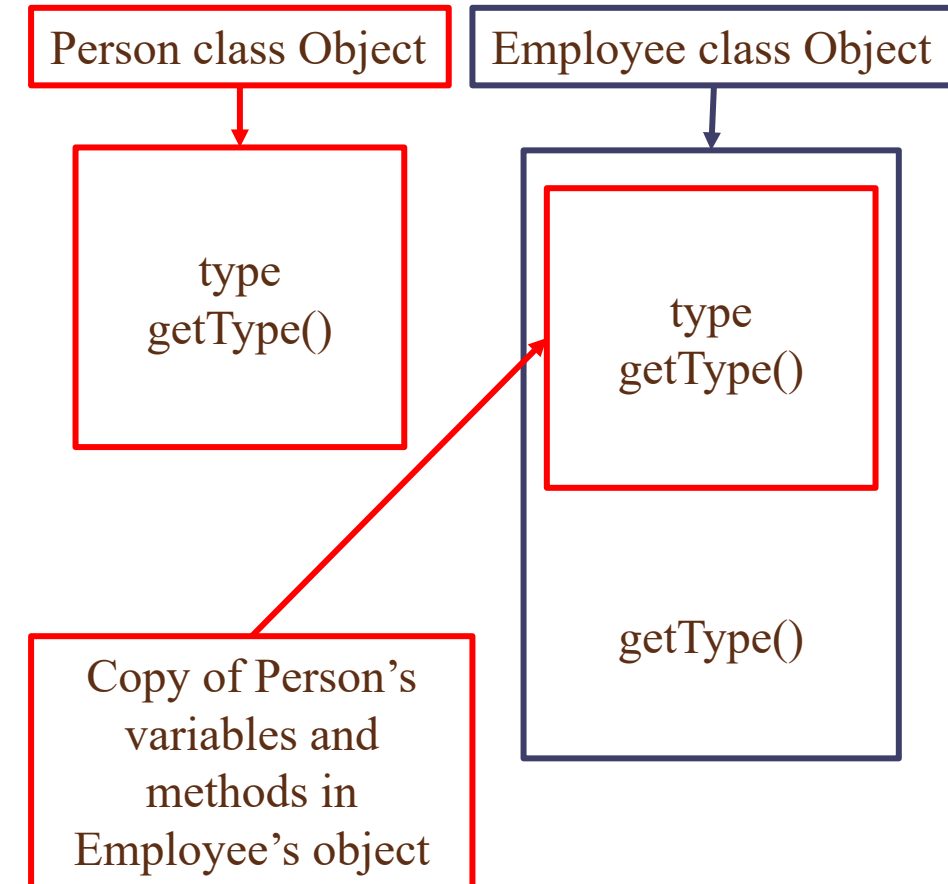
public class Employee extends Person{
    public void getType(){
        type = "Employee";
        System.out.println("This is an "+type);
    }
}
```



Polymorphism

```
public class Main{  
    public static void main(String[] args){  
        Person obj = new Employee();  
        obj.getType();  
    }  
}
```

Output: This is an Employee.



Polymorphism

- Late binding or dynamic binding (run-time binding):
 - Method to be executed is determined at execution time, not compile time.
- Polymorphism: to assign multiple meanings to the same method name.
- The reference variable of a **Person** class can point to any object of the Person or the class **Employee**.
- These reference variables have many forms that is, they are polymorphic reference variables.
- They can refer to objects of their own class or to objects of the classes inherited from their class.



Benefits of Polymorphism

- **Simplicity**
 - Declaring only a reference of a super class, all other subclasses can be handled.
 - Even though the code thinks it is using an object of the base class, the object's class could actually be the base class or any one of its subclasses
 - This makes the code easier to write and easier for others to understand
- **Extensibility**
 - Other subclasses can be added and they will work with existing code.

